

Práctica 3. FreeRTOS: prioridades y núcleos

Ejercicio 1. Circuito

Arma el circuito y verifica que las conexiones correspondan a las indicadas en la Tabla 1 y la Fig. 1. Los dos potenciómetros van con sus extremos a 3V3 y a GND, y su terminal central a la entrada analógica.

Tabla 1. Conexiones del circuito

XIAO ESP32-S3	Se conecta a	Función
3V3	VCC de la pantalla y un extremo de los dos potenciómetros	Alimentación
GND	GND de la pantalla y el otro extremo de los potenciómetros	Referencia común
D5 (GPIO 6)	Terminal central del potenciómetro 1	Entrada analógica. ADC1, canal 5
D4 (GPIO 5)	Terminal central del potenciómetro 2	Entrada analógica. ADC1, canal 4
D8 (GPIO 7)	SDA de la pantalla	Línea de datos I2C
D7 (GPIO 44)	SCL de la pantalla	Línea de reloj I2C

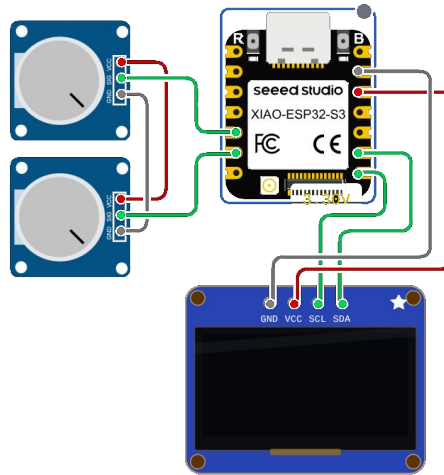


Figura 1. Esquemático

Ejercicio 2. El programa

1. Genera un nuevo proyecto
2. Incluye las librerías de la pantalla /Componentes dentro de la carpeta principal.
3. Incluye el módulo de las animaciones grafico.c y grafico.h dentro de la carpeta /main.
4. Sustituye el contenido del main.c por el programa descargado main.c
5. Compila y carga en la tarjeta. No hay que escribir código nuevo: el trabajo de esta práctica es editar una tabla y tres constantes, y observar el efecto.
6. Identifica algunas partes importantes del programa: el programa principal, el cambio de prioridad, y las funciones de animación.

La pantalla se divide en dos ventanas. En cada una hay un anillo de esferas con un cubo montado, y el potenciómetro de esa ventana lo hace girar en vivo. La figura no está guardada: se calcula con luz y perspectiva cada vez que se dibuja, así que cuesta caro. El equipo no da para mantener las dos ventanas fluidas a la vez, y por eso el programa le da el CPU a la ventana cuya perilla se está moviendo.

Tabla 2. Tareas del programa con su configuración inicial

Tarea	Qué hace	Núcleo	Prioridad
vista1	Dibuja la ventana izquierda. Gira con el potenciómetro 1	0	1
vista2	Dibuja la ventana derecha. Gira con el potenciómetro 2	0	1
control	Lee las perillas, reparte las prioridades y refresca la pantalla	0	1

Ejercicio 3. El programa

En el renglón superior de la pantalla se lee la **situación** que el programa detectó: **SOLO1** (solo se mueve la perilla 1), **SOLO2**, **AMBAS** o **NADIE**. Esa palabra es el renglón de la tabla de prioridades que se está aplicando en ese momento.

En main.c, bajo el rótulo LO QUE EDITAN USTEDES, están la tabla y los tres núcleos. Es lo único que se toca:

```
// Prioridad de cada tarea, de 1 a 5, segun que perilla se mueve.
static const prioridades_t PRIO[N_SIT] = {
    /* SOLO1 */ {1, 1, 5},
    /* SOLO2 */ {1, 1, 5},
    /* AMBAS */ {1, 1, 5},
    /* NADIE */ {1, 1, 5},
};

// Nucleo de cada tarea. Solo compiten las que comparten core.
#define SIN_AFINIDAD tskNO_AFFINITY
#define CORE_VISTA1 0
#define CORE_VISTA2 0
#define CORE_CONTROL 0
```

Prueba distintas combinaciones siguiendo este procedimiento:

1. Edita la tabla o los núcleos.
2. Compila, carga y gira las perillas 15 o 20 segundos. Una a la vez, las dos juntas, y sin girar ninguna.
3. Identifica qué hace **cada** ventana, en cada una de las cuatro situaciones. Responde al instante, responde con retraso, va a tirones, no responde.
4. Deja la configuración que consideres mejor y justifícala.

Reporte

Elabora un reporte siguiendo el formato de la plantilla Reporte_plantilla.docx proporcionada por el profesor. Los siguientes puntos te pueden ayudar a redactarlo.

1. **Objetivos.** Escribe los objetivos que identificaste en la práctica.
2. **Resultados y análisis.** Incluye la tabla con lo observado en los cinco experimentos, y apóyate en estas preguntas:
 - ¿Qué combinación de núcleos y prioridades es la más eficiente y por qué?
 - ¿Dentro del código, existe otra forma de hacer más fluidas las animaciones aparte de cambiar la prioridad y el núcleo?
 - Investiga si es posible cambiar el núcleo en tiempo de ejecución como sí se puede con las prioridades.
 - ¿Por qué control tiene que ser la tarea de mayor prioridad de su núcleo? Usa el hecho de que es la única que puede volver a subir a una tarea degradada.
 - El proceso de control quita recursos a los procesos que comparten su núcleo. ¿Se puede equilibrar mejor las cargas partiendo el procesamiento de cada ventana y del control en sub funciones?
3. **Conclusiones.** Puedes apoyarte en estas preguntas:
 - ¿En qué se parece este programa a un sistema operativo con varias ventanas abiertas?
 - La prioridad se cambia en vivo con `vTaskPrioritySet()`, pero el núcleo se decide al crear la tarea y ya no se cambia. ¿Qué consecuencia tuvo eso en el experimento 3?
 - ¿Qué tareas de un sistema real no deberían quedar nunca por debajo de otra, y por qué?